Rational Matrix Policy Optimization for Rational Feed-Forward Language Models

Anonymous authors
Paper under double-blind review

Abstract

1 Method

1.1 Rational local-basis feed-forward block

We replace the standard gated feed-forward block with a single-branch Rational Local Basis (RLB) block. Let $x \in \mathbb{R}^d$ be the residual-stream input to a Transformer feed-forward layer. The input projection maps into a hidden space whose channels are partitioned into G groups of width m:

$$z = A_l x, \tag{1}$$

$$z_g = \mathrm{group}_g(z), \qquad g = 1, \ldots, G, \tag{2}$$

where $A_l = W_{\mathrm{in},l}$. Each group is normalized by its root-mean-square radius,

$$r_g = \sqrt{m^{-1}\|z_g\|_2^2 + \epsilon}, \qquad u_g = z_g / r_g, \tag{3}$$

and then passed through a learned rational function,

$$h_g = r_g R_{l,g}(u_g). \tag{4}$$

The group outputs are concatenated and projected back to the residual stream,

$$y = B_l \,\mathrm{concat}_{g=1}^{G}(h_g), \tag{5}$$

where $B_l = W_{\mathrm{out},l}$. In the current implementation, $R_{l,g}$ is a base rational curve plus trainable local odd/bump basis terms around fixed centers. The block has no GLU branch, no hidden SiLU branch, and no multiplicative value-gate split. This is important: the optimizer is not optimizing a standard SwiGLU block under a different name; it is optimizing the explicit factorization

$$A_l \;\longrightarrow\; \{R_{l,g}\}_{g=1}^{G} \;\longrightarrow\; B_l. \tag{6}$$

1.2 Positive gauge symmetry

The group normalization makes each RLB group positively homogeneous. For any positive group scale $a_g > 0$,

$$z'_g = a_g z_g, \qquad r'_g = a_g r_g, \qquad u'_g = u_g, \qquad h'_g = a_g h_g. \tag{7}$$

Therefore the matrix reparameterization

$$A'_{l,g} = a_g A_{l,g}, \qquad B'_{l,g} = B_{l,g}/a_g \tag{8}$$

preserves the represented layer function exactly up to floating-point error. In block form, with $D_l(a)$ block diagonal and block $a_g I_m$,

$$A'_l = D_l(a) A_l, \qquad B'_l = B_l D_l(a)^{-1}. \tag{9}$$

A generic optimizer is not invariant to this reparameterization: the gradients, parameter norms, and adaptive second moments change even though the function does not. The central design principle of MatrixPolicy is therefore to spend updates on useful function motion while controlling unnecessary motion along this gauge direction.

1.3 Parameter roles

The RLB block exposes three distinct parameter roles. The input matrix A_l chooses the domain seen by each rational group. The rational coefficients set the local nonlinear shape and derivative profile. The output matrix B_l recombines rational features back into the residual stream. MatrixPolicy keeps these roles separate. For trainable parameters θ, the optimizer partition is

$$\theta = \theta_{\text{backbone}} \cup \theta_{\text{coeff}} \cup \{M_{l,\text{in}} = A_l, M_{l,\text{out}} = B_l\}_{l=1}^{L}. \tag{10}$$

In the current real-corpus configuration,

$$\theta_{\text{backbone}} \leftarrow \text{AdamW}, \tag{11}$$
$$\theta_{\text{coeff}} \leftarrow \text{coefficient updates in the RLB wrapper}, \tag{12}$$
$$M_{l,\text{in/out}} \leftarrow \text{RationalMatrixPolicyOptimizer}, \tag{13}$$

followed by an exact post-step gauge rebalance. Thus MatrixPolicy is not a whole-model optimizer. It is an RLB-local matrix policy embedded inside an otherwise standard Transformer optimizer.

1.4 Role-depth matrix policy

Let $d_l = l/(L-1)$ be normalized depth. The input and output matrices are given different depth-dependent role factors,

$$\rho_{\text{in}}(l) = \text{clip}\left(1 - 0.50(d_l - 0.5),\ 0.55,\ 1.40\right), \tag{14}$$
$$\rho_{\text{out}}(l) = \text{clip}\left(1 + 1.00(d_l - 0.5),\ 0.55,\ 1.40\right). \tag{15}$$

Earlier layers are biased toward input-domain selection; later layers are biased toward output-feature recombination. The Adam-style role multiplier is

$$a_{\text{role}}(l, r) = \max\left(0.10,\ 1 + \alpha_{\text{role}}(\rho_r(l) - 1)\right), \qquad \alpha_{\text{role}} = 1.20, \tag{16}$$

for $r \in \{\text{in}, \text{out}\}$. This does not alter the global learning rate schedule. The base schedule η_t is shared with the AdamW and Muon controls.

1.5 AdamW–Muon matrix mixture

For each RLB matrix role $M_{l,r}$, MatrixPolicy combines an AdamW-style update with a short early Muon-style matrix update. The Muon fraction is

$$\mu(l, r, t) = \text{clip}\left(\mu_{\text{base}}(t)\rho_r(l)s_{\text{stat}}(l, t),\ 0,\ 0.75\right). \tag{17}$$

The base window starts at training progress 0.02, reaches full strength by 0.12, decays from 0.20 to 0.36, and then returns to zero. The effective update is

$$M_{l,r}^{t+1} = M_{l,r}^{t} + \Delta_{\text{AdamW}}(M_{l,r}; \eta_t a_{\text{mat}}(l, r, t)[1 - \mu(l, r, t)]) + \Delta_{\text{Muon}}(M_{l,r}; \eta_t a_{\text{muon}}\mu(l, r, t)), \tag{18}$$

where

$$a_{\text{mat}}(l, r, t) = \text{clip}\left(3.0\, a_{\text{role}}(l, r)a_{\text{stat}}(l, r, t),\ 0.40,\ 4.0\right), \qquad a_{\text{muon}} = 1.0. \tag{19}$$

The motivation is that early orthogonalized matrix movement can quickly select useful rational domains and output features, while the decay prevents late training from being dominated by generic matrix pressure.

1.6 On-policy group statistics

Before each child optimizer step, the wrapper records per-group relative pressure statistics,

$$p_{\text{in},g} = \frac{\text{rms}(\nabla A_{l,g})}{\text{rms}(A_{l,g})}, \qquad p_{\text{out},g} = \frac{\text{rms}(\nabla B_{l,g})}{\text{rms}(B_{l,g})}, \qquad p_{\text{rat},g} = \text{rms}(\nabla \theta_{\text{coeff},g}). \tag{20}$$

Their exponential moving averages are denoted $q_{\text{in},g}$, $q_{\text{out},g}$, and $q_{\text{rat},g}$. Two derived quantities drive the group-stat policy:

$$\pi_g = \log q_{\text{in},g} - \log q_{\text{out},g}, \tag{21}$$

$$\alpha_g = \log q_{\text{rat},g} - \frac{1}{2}\left(\log q_{\text{in},g} + \log q_{\text{out},g}\right). \tag{22}$$

Here π_g is input-output pressure imbalance and α_g is rational-shape activity relative to the surrounding matrices.

1.7 Group-stat matrix preconditioning

The best current real-corpus configuration uses a mild group-stat matrix preconditioner. For each group,

$$c_g = c_{\text{gain},g} c_{\text{pressure},g} c_{\text{activity},g}, \qquad c_g \leftarrow c_g / \operatorname{geomean}(c), \qquad c_g \leftarrow \operatorname{clip}(c_g, 0.75, 1.35). \tag{23}$$

The gain term balances derivative or output activity,

$$c_{\text{gain},g} = \left(\frac{\operatorname{geomean}(k)}{k_g}\right)^{0.20}, \tag{24}$$

where k_g is derivative RMS for A_l and output RMS for B_l. The pressure term is

$$c_{\text{pressure},g} = \exp(0.10\, d_{\text{pressure},g}), \tag{25}$$

with $d_{\text{pressure},g} = -\pi_g$ for A_l and $d_{\text{pressure},g} = +\pi_g$ for B_l. The activity term damps overactive rational-shape movement,

$$c_{\text{activity},g} = \exp\left[-0.20\ \text{ReLU}\left((\alpha_g - 0.05)/0.45\right)\right]. \tag{26}$$

This group-stat policy is active from progress 0.02 to 0.30. It scales A_l gradients group-row-wise and B_l gradients group-column-wise.

1.8 Function-preserving gauge rebalance

After the child optimizers step, MatrixPolicy applies an exact RLB gauge correction. Let

$$n_{\text{in},g} = \operatorname{rms}(A_{l,g}), \qquad n_{\text{out},g} = \operatorname{rms}(B_{l,g}), \qquad c_g^{\text{gauge}} = \log n_{\text{in},g} - \log n_{\text{out},g}. \tag{27}$$

The target gauge combines rational derivative/output statistics with pressure,

$$\tau_g = \frac{1}{2}\left(\log d_g^{\text{gain}} - \log o_g^{\text{gain}}\right) + \beta_{\text{pressure}}(\log q_{\text{in},g} - \log q_{\text{out},g}). \tag{28}$$

The applied log-scale step is clipped,

$$\ell_g = \frac{1}{2}(\tau_g - c_g^{\text{gauge}}), \qquad \ell_g \leftarrow \operatorname{clip}(s(t,l) a_g^{\text{activity}} \ell_g, -\ell_{\max}, \ell_{\max}), \tag{29}$$

and the matrices are reparameterized as

$$A_{l,g} \leftarrow \exp(\ell_g) A_{l,g}, \qquad B_{l,g} \leftarrow B_{l,g} / \exp(\ell_g). \tag{30}$$

This step is function-preserving. Its purpose is to choose a better-conditioned representative of the same RLB function rather than to directly reduce loss by a learning-rate change.

1.9 Step summary

One MatrixPolicy optimizer step is:

1. record live RLB pressure and activity statistics;
2. apply optional group-stat scaling to RLB matrix gradients;
3. update backbone parameters with AdamW;
4. update rational coefficient parameters through the RLB wrapper;
5. update A_l and B_l with the role-depth AdamW–Muon matrix policy;
6. apply the exact post-step gauge rebalance.

The design is deliberately local to rational blocks. It uses the same global base learning-rate schedule as the controls and derives its additional behavior from RLB roles, layer depth, live group statistics, and the positive gauge symmetry.